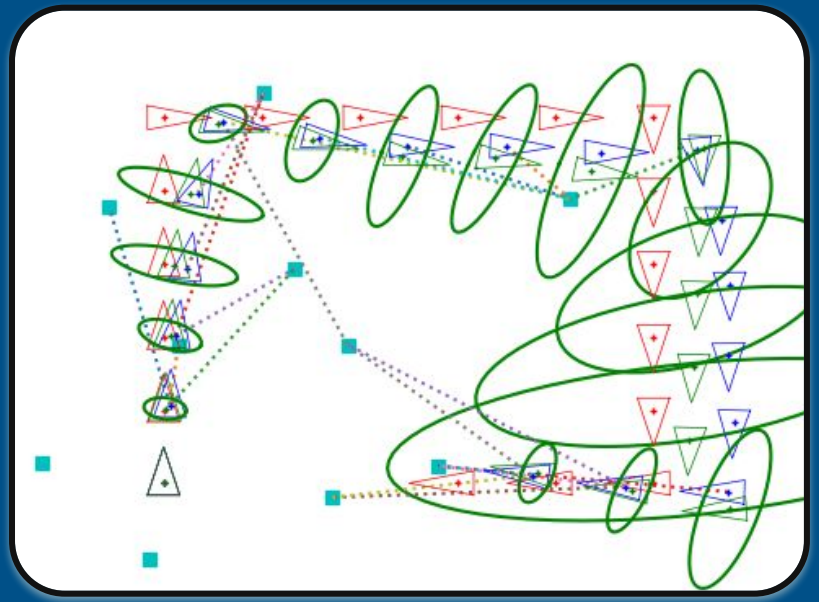
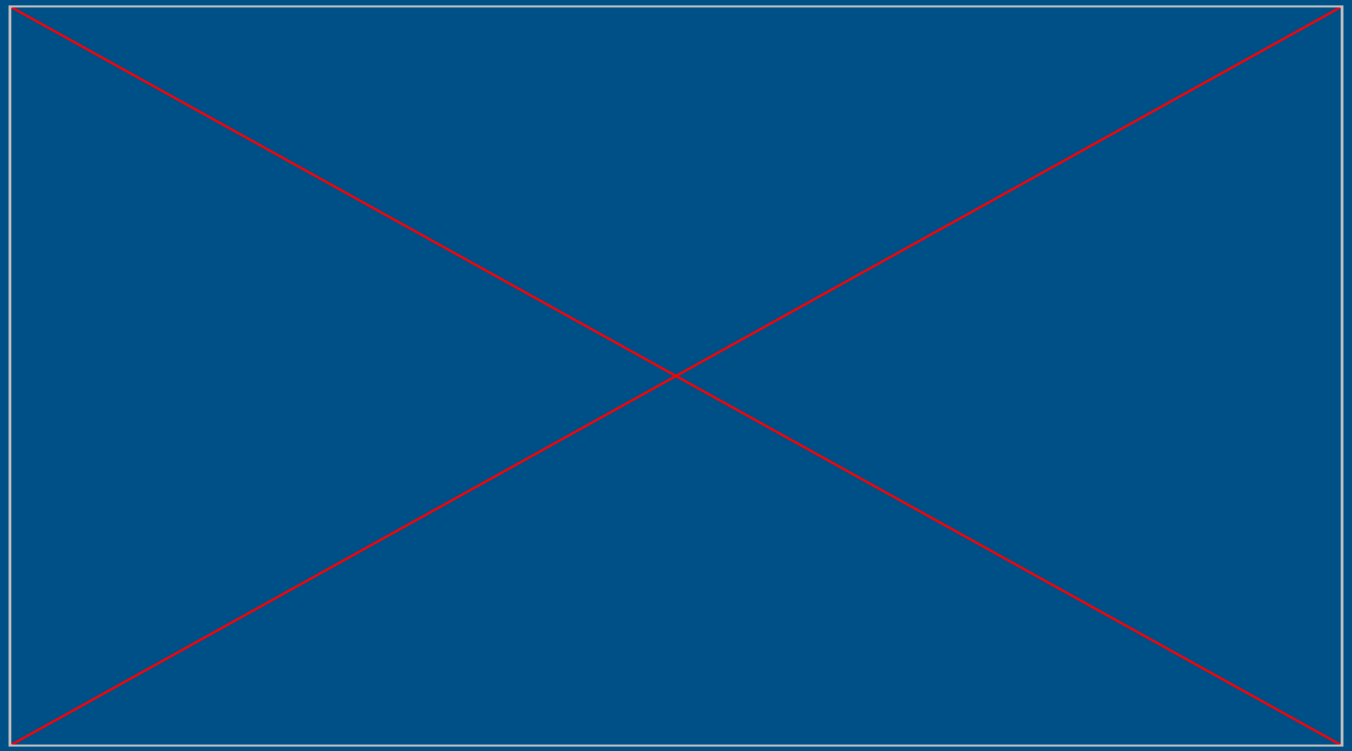
I402: Principios de la Robótica Autónoma

Clases Prácticas
Tutorial 1



Universidad de SanAndrés



¿Quienes somos?



Tadeo Casiraghi

tcasiraghi@udesa.edu.ar



Bautista Chasco

bchasco@udesa.edu.ar

¿Qué vamos a hacer en las prácticas?

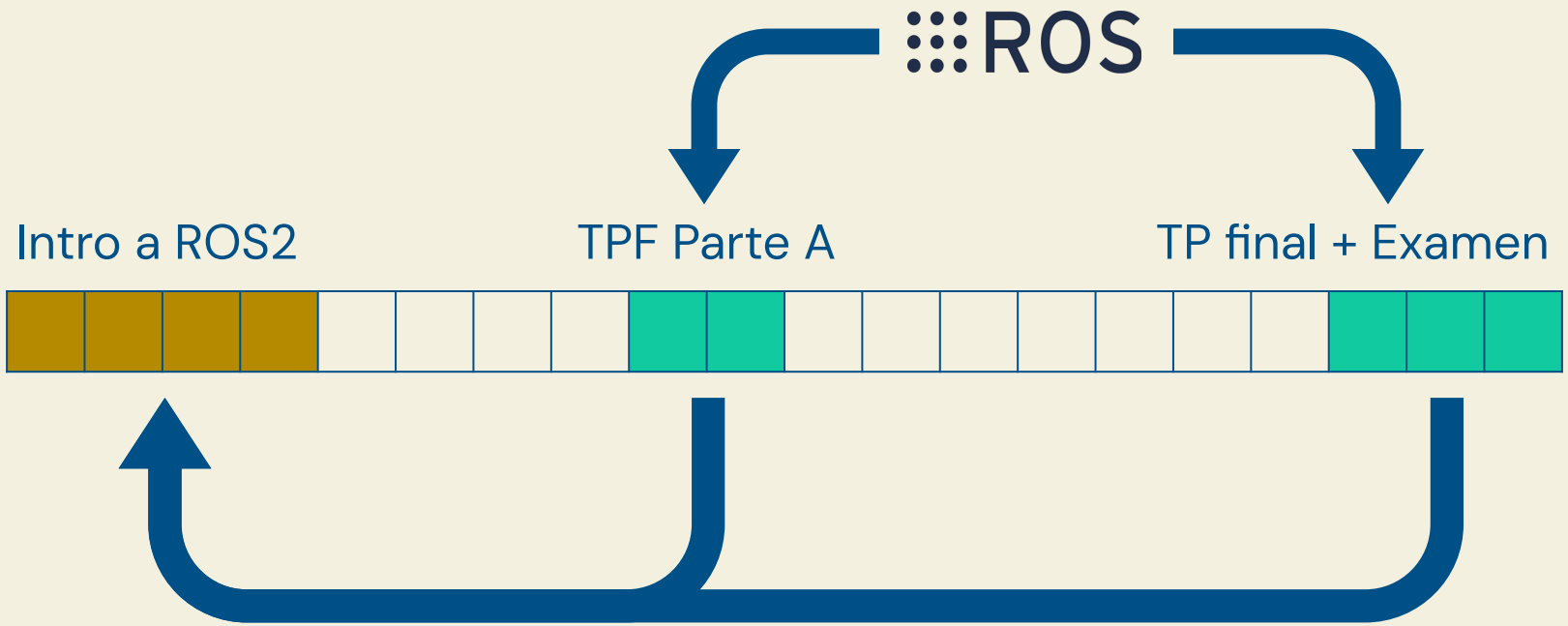


20 semanas

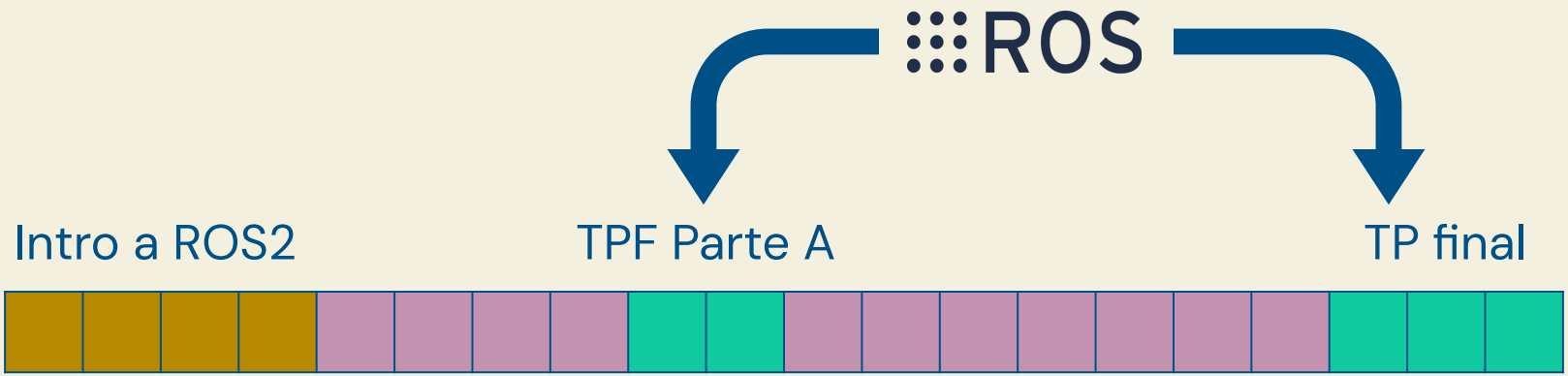
¿Qué vamos a hacer en las prácticas?



¿Qué vamos a hacer en las prácticas?



¿Qué vamos a hacer en las prácticas?



Repaso y exploración de lo visto en teórica
+ aplicación en simulación con ROS2

Introducción a ROS

1402



1. ¿Porqué ROS?
2. ¿Qué es ROS?
3. En nuestro caso...
4. Compatibilidad
5. Tarea para la casa

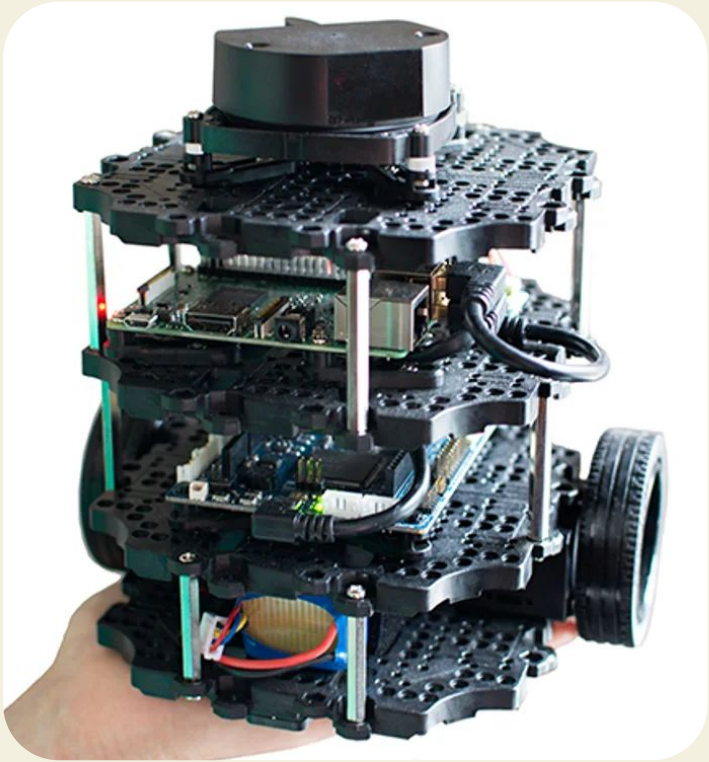
1. ¿Porqué ROS?
2. ¿Qué es ROS?
3. En nuestro caso...
4. Compatibilidad
5. Tarea para la casa

Tiene:

- Procesador/CPU
- Ruedas → capacidad de movimiento
- Algún sensor

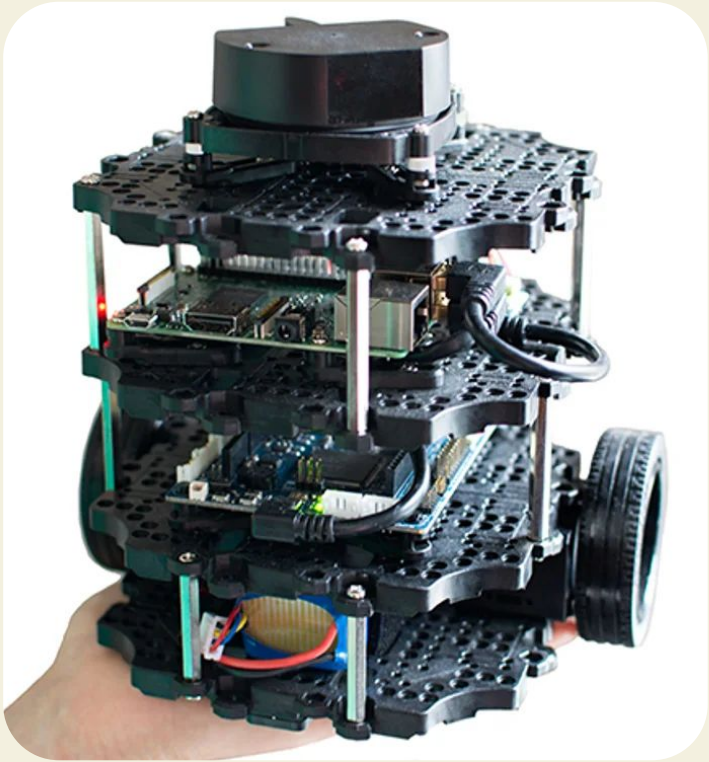
Quiero:

- Programar un algoritmo de control para que recorra un ambiente y llegue a algún destino de manera autónoma



Hago un código que en todo momento:

- Lea los datos del sensor
- Interprete lo que ve del sensor
- Decida qué hacer
- Ejecute los comandos de movimiento
- Mida que tan bien se movió en base a lo que le pedí
- Calcule el mejor camino para llegar al objetivo
- Se comunique con una PC externa para monitorear su desempeño



Hago un código que en todo momento:

- Lea los datos del sensor
- Interprete lo que ve del sensor
- Decida qué hacer
- Ejecute los comandos de movimiento
- Mida que tan bien se movió en base a lo que le pedí
- Calcule el mejor camino para llegar al objetivo
- Se comunique con una PC externa para monitorear su desempeño

Esto implica que tengo que lidiar con:

- ➔ Interrupciones por lectura de datos
- ➔ Procesar datos nuevos cada vez
- ➔ Esperar a que haya datos nuevos
- ➔ Comunicarme con el hardware
- ➔ Ante cada movimiento y lectura de datos procesar y evaluar
- ➔ Estar al tanto del movimiento en todo momento
- ➔ Networking entre PC y procesador del robot

Esto implica que tengo que lidiar con:

- ➔ Interrupciones por lectura de datos
- ➔ Procesar datos nuevos cada vez
- ➔ Esperar a que haya datos nuevos
- ➔ Comunicarme con el hardware
- ➔ Ante cada movimiento y lectura de datos procesar y evaluar
- ➔ Estar al tanto del movimiento en todo momento
- ➔ Networking entre PC y procesador del robot

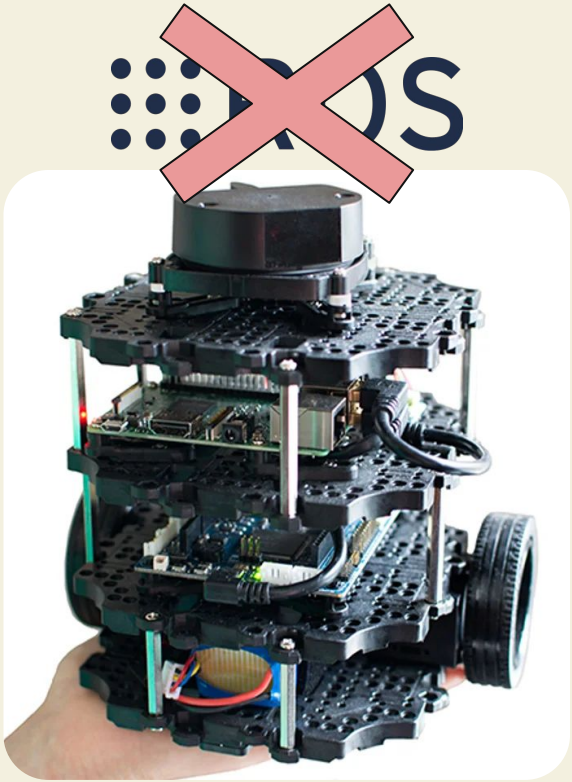
Si bajamos más a lo que implica para nuestro código ahora tenemos que lidiar con:

- Threading para manejar todos los procesos a destiempo sin bloquear el algoritmo
- Comunicación entre computadoras a través de alguna red, tambien a destiempo
- Flujo de datos entre sensores y actuadores y el procesador

1. ¿Porqué ROS?
2. **¿Qué es ROS?**
3. En nuestro caso...
4. Compatibilidad
5. Tarea para la casa

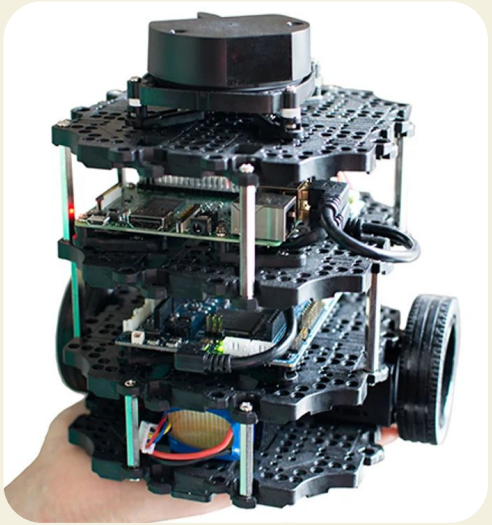
ROS

Robot Operating System



ROS

Robot Operating System





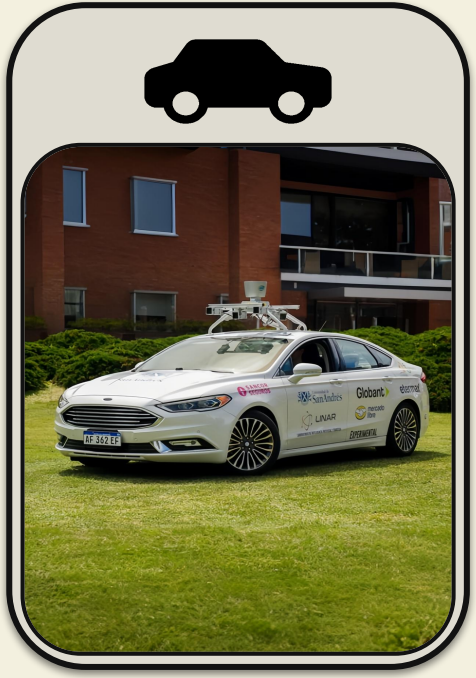
Se va a encargar de manejar:

- Todo tipo de threading para manejar procesos a destiempo sin bloquear el algoritmo
- Comunicación entre computadoras a través de alguna red, tambien a destiempo
- Flujo de datos entre sensores y actuadores y el procesador

Pero además tengo:

- Software de simulación física de mi robot, sus sensores y el entorno
- Visualización de los datos de mi robot
- Grabación integrada de todos los datos de mis ensayos para una posterior evaluación o reproducción en “tiempo real”
- Paquetes pre-armados por terceros
- Aislación de paquetes para una mayor facilidad de modificaciones
- Abstracción de lenguaje (Python y C++)
- Comunidad activa de usuarios

ROS

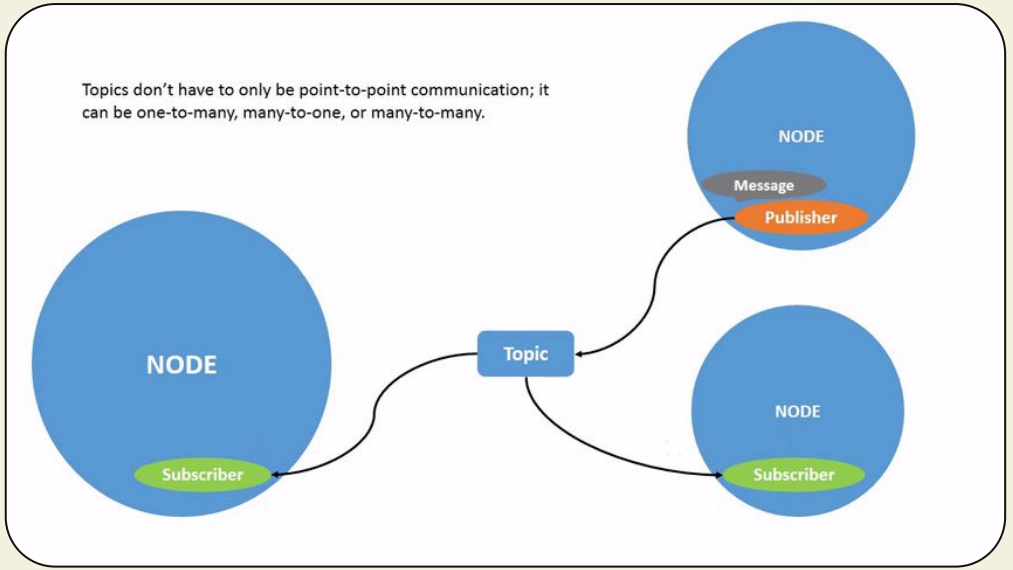




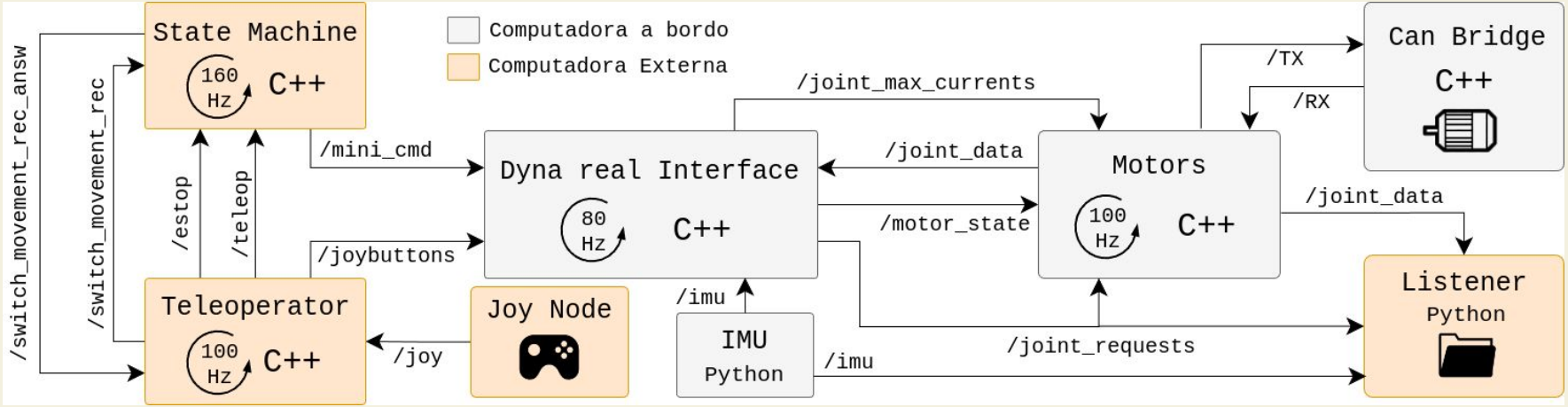


ROS: Arquitectura

- Nodo
  - Corre procesos
  - Puede publicar y suscribirse a topics
- Topics
  - Canales para mandar mensajes
- Mensajes
  - Datos que los nodos usan para comunicarse

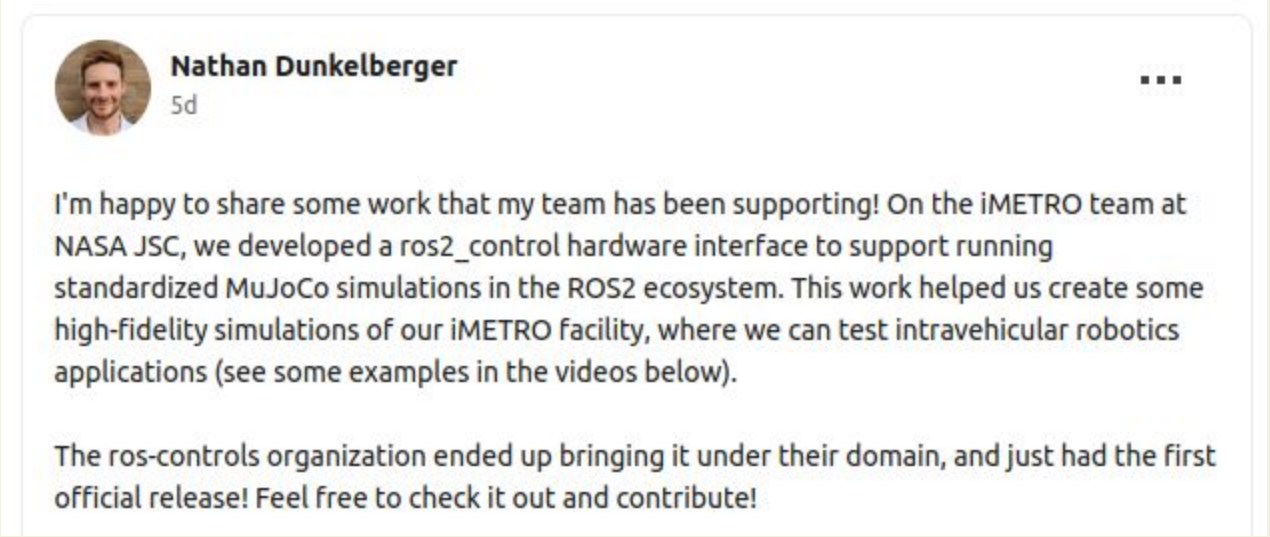


ROS


Computadora a bordo
Computadora Externa
State Machine
160 Hz
C++
Teleoperator
100 Hz
C++
Dyna real Interface
80 Hz
C++
Motors
100 Hz
C++
Joy Node
IMU
Python
Can Bridge
C++
Listener
Python
/switch_movement_rec_answ
/switch_movement_rec
/estop
/teleop
/mini_cmd
/joybuttons
/joy
/joint_max_currents
/joint_data
/motor_state
/TX
/RX
/joint_data
/imu
/imu
/joint_requests

ROS: Arquitectura

- Nodo
  - Corre procesos
  - Puede publicar y suscribirse a topics
- Topics
  - Canales para mandar mensajes
- Mensajes
  - Datos que los nodos usan para comunicarse
- Servicios
- Acciones
- Rosbag
- Nodos composables
- QOS de mensajes
- Parámetros
- RTO
- URDF
- Capa de DDS y variantes
- Memory Allocation
- ROS2 Tracing
- Simuladores
- Mucho mucho más



Nathan Dunkelberger
5d

I'm happy to share some work that my team has been supporting! On the iMETRO team at NASA JSC, we developed a ros2_control hardware interface to support running standardized MuJoCo simulations in the ROS2 ecosystem. This work helped us create some high-fidelity simulations of our iMETRO facility, where we can test intravehicular robotics applications (see some examples in the videos below).

The ros-controls organization ended up bringing it under their domain, and just had the first official release! Feel free to check it out and contribute!

1. ¿Porqué ROS?
2. ¿Qué es ROS?
3. **En nuestro caso...**
4. Compatibilidad
5. Tarea para la casa

Versión



Simulador



Paquetes



ROS2 Humble → Ojo con la versión de la documentación que están usando



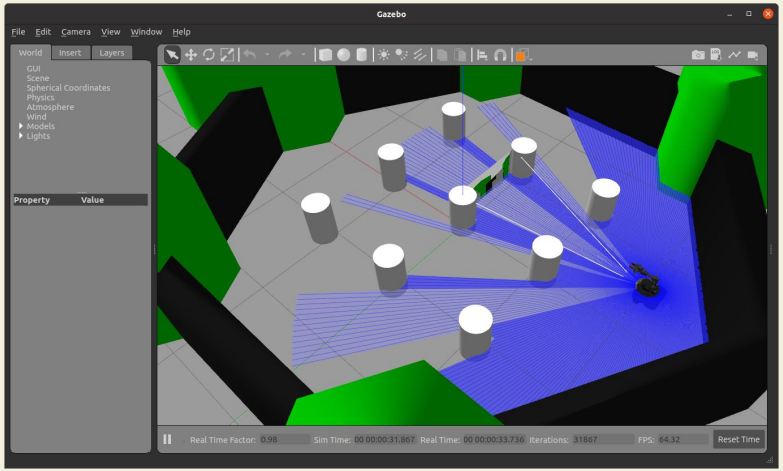
GAZEBO

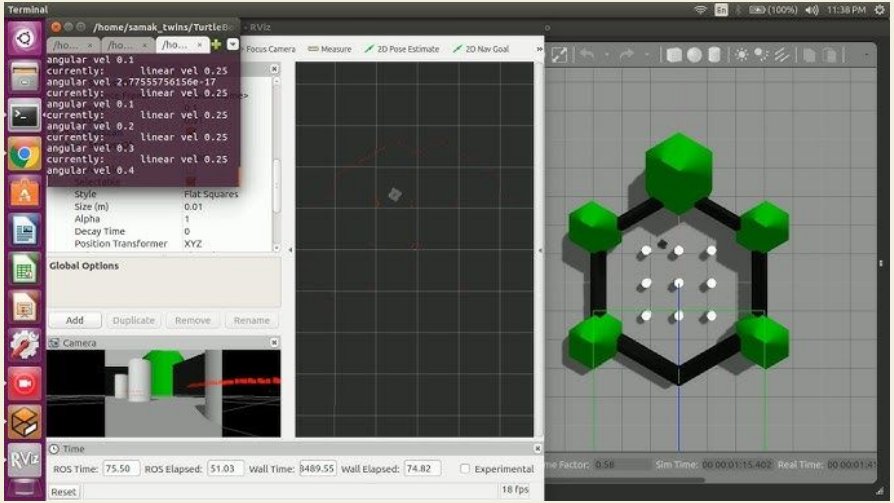
YAHBOOM

https://docs.ros.org/en/humble/index.htm

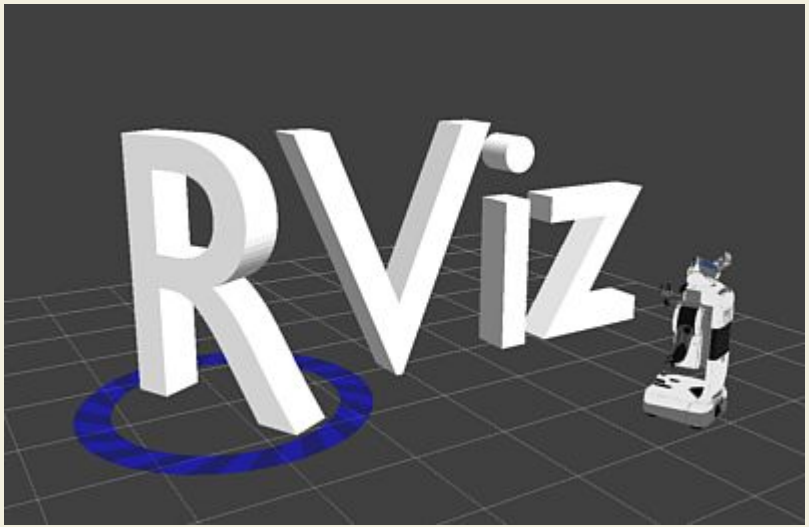
https://emanual.robotis.com/docs/en/platform/turtlebot3/simulation/

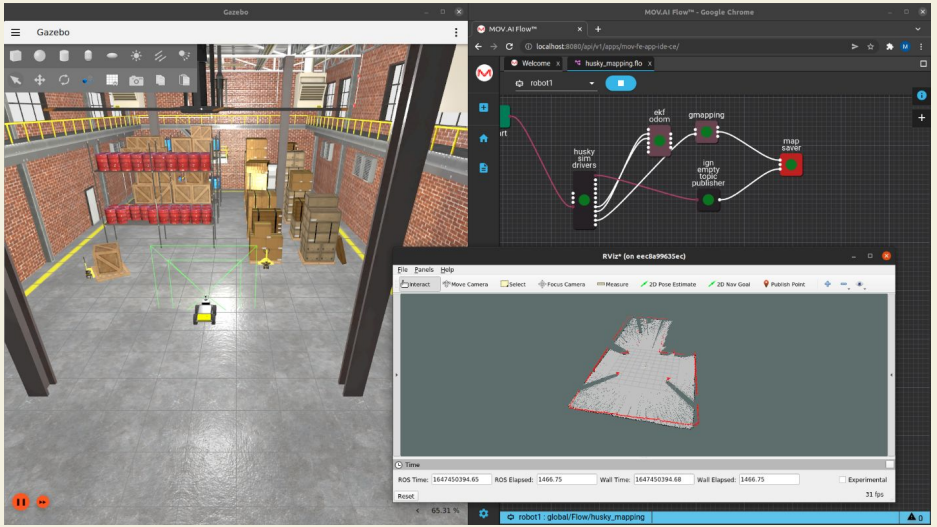
Gazebo – Simulo el robot y el entorno





RVIZ – ¿Qué ve el robot?





1. ¿Porqué ROS?
2. ¿Qué es ROS?
3. En nuestro caso...
4. **Compatibilidad**
5. Tarea para la casa

Open source...

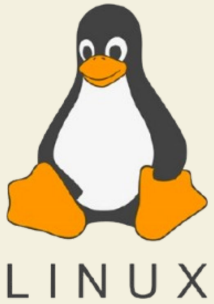
- 100% de compatibilidad con Linux
- Especificamente Ubuntu (cada versión de ROS es compatible con algunas versiones de Ubuntu en particular)
- Open Robotics está abriéndose a nuevos OS lentamente



- Hay muchos desarrollos para trabajar ROS usando otros OS's!

- No son triviales...

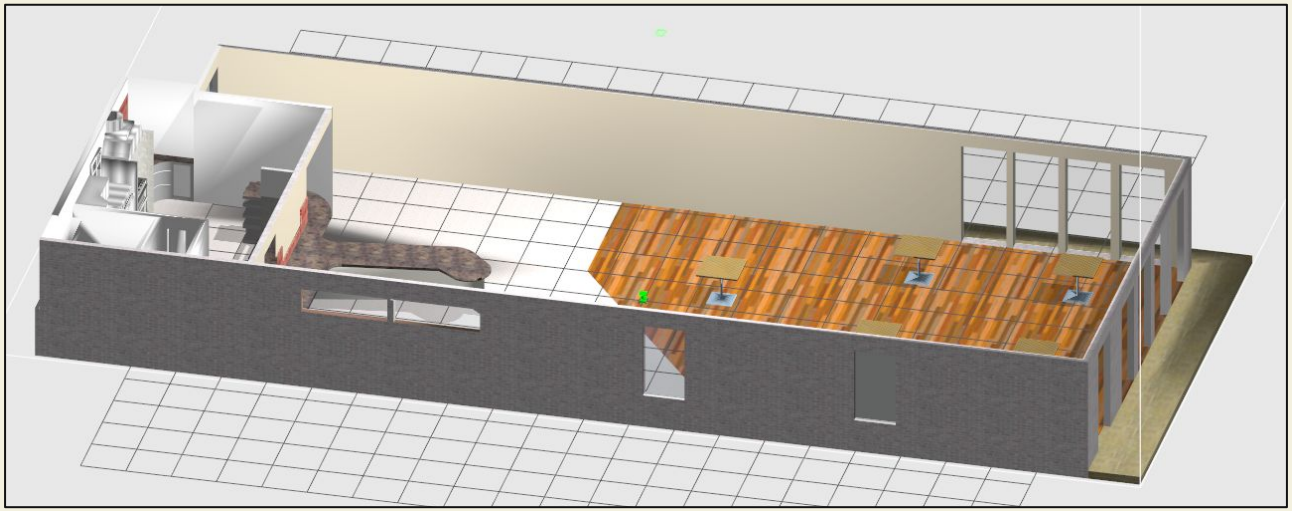






1. ¿Porqué ROS?
2. ¿Qué es ROS?
3. En nuestro caso...
4. Compatibilidad
5. **Tarea para la casa**

Tarea para la semana que viene



Limitantes

- Gazebo opera con el GPU
- Precisa GPU de al menos 1Gb (verificar requisito)
- La interacción con el GPU es variable según la arquitectura del microprocesador (ARM, AMD, Intel) -> cada alternativa está adaptada a arquitecturas de chip específica, siendo ARM (MacOS, chip de línea M) la menos disponible



Linux

En campus podrán encontrar un DockerFile y un archivo denominado “container.sh”.

container.sh es un helper que se encarga de correr todo y les sirve de método de entrada simplificada al docker.

1. Se descargan los dos archivos y los ponen en un directorio que quieran, idealmente no estan tirados en “Downloads”.
2. Dentro de ese directorio usan:

```
./container.sh start
```

3. Después de que buildee:

```
./container.sh enter
```

Linux

Una vez que hicieron todo eso, y dentro del docker, pueden probar que todo esté funcionando bien con lo siguiente:

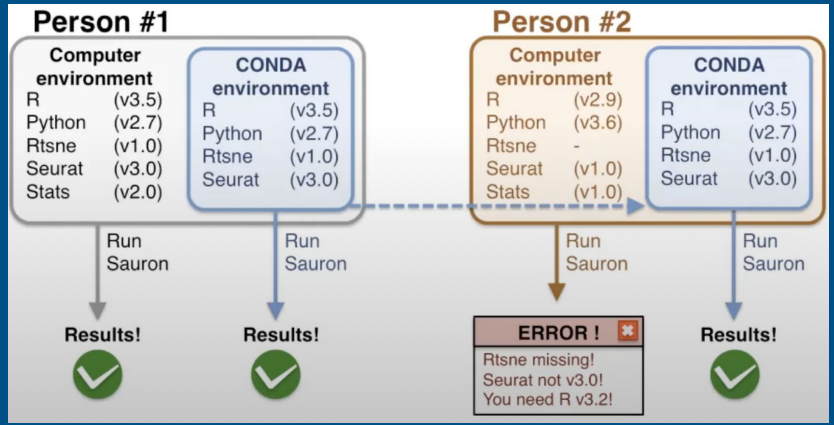
```
ros2 launch yahboom_rosmaster_gazebo rosmaster_gazebo_fortress.launch.py \
  world:="$(ros2 pkg prefix yahboom_rosmaster_gazebo)/share/yahboom_rosmaster_gazebo/worlds/cafe.world"
```

Les debería aparecer el mundo simulado de una cafetería.

Robostack

- Un conjunto de paquetes Conda desarrollados por Open Robotics
- Virtual environment
- Ajusta la instalación de paquetes según OS y chip
- Comunidad muy activa en Git
- Falla mucho pero una vez instalado es altamente reproducible





Robostack

- https://github.com/bchax/yahboom_rosmaster/blob/main/yahboom_robostack_M_silycon/SETUP.md

Windows

5 opciones...

1. Prueban con WSL y el docker (Puede no funcionar)
2. Prueban con Robostack (Puede no funcionar)
3. Hacen una partición y se instalan Ubuntu (Va a funcionar, y para este punto ya deberían)
4. Usan las computadoras del laboratorio
5. Hacen todo el código sin jamás probarlo antes de subir al campus y se entregan a la suerte de los dioses de la programación

Tarea para la semana que viene

